

operation will trigger all the lined up transformations on the base RDD and then execute the action operation on the last RDD.

Map transformation: Map applies a function to each element of the RDD. The following code checks the length of each line.

```
val length = lines.map(s => s.length)
length.collect()
```

Reduce action: Reduce aggregates the elements according to key value. It is applied to the output of the Map function. The following code calculates the sum total of characters in the file.

```
val totalLength = length.reduce(a, b) => a + b)
```

The DataFrame API

DataFrame is a distributed data collection organised into columns that are similar to a relational database system. DataFrame can be created from Hive tables, structured data files, external databases or existing RDDs. The DataFrame API uses a schema to describe the data, allowing Spark to manage the schema and only pass data between nodes. This is more efficient than using Java serialisation. It is helpful when performing computations in a single process, as Spark can serialise the data into off-heap storage in a binary format and then perform many transformations directly on this, reducing the garbage-collection costs of constructing individual objects for each row in the data set. Because Spark understands the schema, we don't need to use Java serialisation to encode the data. The following code will explore some functions related to DataFrames.

First, create a *sparkcontext* object, as follows:

```
val sqlContext = new org.apache.spark.sql.SQLContext(sc)
```

Then, read an external JSON object and store it in DataFrame *dfs*. Next, show and print it, using the following code:

```
val dfs = sqlContext.read.json("employee.json")
dfs.show()
dfs.printSchema()
```

DataSet API

DataSet API has encoders that translate between JVM representations (objects) and Spark's internal binary format. Spark has built-in encoders, which are powerful as they generate byte code to mix with off-heap data, and provide on-demand access to individual attributes without having to de-serialise an entire object. Moreover, the DataSet API is designed to work well with Scala. When working with Java objects, it is important that they are fully JavaBean-

compliant. The following code will explore some basic functions:

```
val sc = new SparkContext(conf)
val sqlContext = new SQLContext(sc)
import sqlContext.implicits._
val sampleData: Seq[ScalaPerson] = ScalaData.sampleData()
val dataset = sqlContext.createDataset(sampleData)
```

The Scala vs Python debate

Scala and Python are both powerful and popular among data scientists, many of whom learn and work with both languages. Scala is faster and moderately easy to use, while Python is slower but very easy to use. But Python usually ranks second because of the following reasons:

1. Scala is usually 10 times faster than Python for processing. Python code needs a lot of translation and converting, which makes the program a bit slow. Hence, there is a performance overhead. Scala has an edge in performance when there are fewer processor cores.
2. Scala is better for concurrency due to its ability to easily integrate across several databases and services. It has asynchronous libraries and reactive cores. Python doesn't provide heavyweight processes to fork for multi-threading or parallel computing.
3. The Scala programming language has several existential types, macros and implicits. Its advantages are evident when using these powerful features in important frameworks and libraries. Scala is the best choice for the Spark streaming feature because Python Spark streaming support is not advanced and mature like Scala.

Scala has slightly fewer machine learning and natural language processing libraries than Python. The library has only a few algorithms but they are sufficient for Big Data applications. Scala lacks good visualisation and local data transformations. Nevertheless, it is preferred since Python increases portability for more issues and bugs, as translation is tough. That Scala is the winning combination of both object-oriented and functional programming paradigms might be surprising to beginners and they could take some time to pick up the new syntax. Scala programming might be a difficult language to master for Apache Spark, but the time spent on learning it is worth the investment. 

References

- [1] spark.apache.org/
- [2] www.tutorialspoint.com/apache_spark/
- [3] www.kdnuggets.com/2015/06/introduction-big-data-apache-spark.html
- [4] www.tutorialspoint.com/scala/

By: Preet Gandhi

The author is an avid Big Data and data science enthusiast. She can be reached at gandhipreet1995@gmail.com